LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

03

Linux Users, Groups and Permissions

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 Understanding Linux Users
02 User and Group Files
03 Managing Users
04 Linux Groups
05 File Permissions
06 Numeric Permissions
07 Changing Permissions: chmod
08 Ownership: chown and chgrp
09 SUID, SGID and Sticky Bit
10 Default Permissions: umask
11 Access Control Lists
12 sudo and su
13 DevOps Scenarios
14 Interview Q&A
15 Takeaways and Exercises

Ravi Tomar
Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux
github.com/rv-ops/linux-interview-preparation-guide

CHAPTER OBJECTIVE

Understand how Linux manages user accounts, groups, file ownership and permissions. Learn how to grant or restrict access safely, manage administrative privileges, and troubleshoot common permission issues in production environments.

1 Understanding Linux Users

Linux is a multi-user operating system. Multiple users can access the same system, but each user can have different permissions.

Every Linux process runs under a user identity. This helps Linux control which files, directories and resources a process can access.

There are three common types of users:

User type	Purpose
Root user	Full administrative access to the system
System users	Run services and background processes
Regular users	Used by administrators and other people

Root User

The root user is the superuser in Linux. It has UID `0` and can perform almost any administrative operation.

```
# Check the current user
whoami

# Check user ID and group information
id

# Switch to root
sudo -i
```

IMPORTANT

Avoid using root for everyday activities. Use a regular account with `sudo` when administrative access is required.

User ID (UID)

Every Linux user has a numeric User ID.

- Root normally has UID `0`.
- Regular users commonly have UIDs starting from `1000` on many distributions.
- System users commonly have lower UIDs.

```
# Check a user's UID
id username
```

2 Important User and Group Files

Linux stores user and group information in specific system files.

File	Purpose
`/etc/passwd`	User account information
`/etc/shadow`	Password hashes and password-aging information
`/etc/group`	Group information
`/etc/gshadow`	Secure group administration information

```
# View user accounts
cat /etc/passwd
```

Example entry:

```
john:x:1001:1001:John:/home/john:/bin/bash
```

The fields represent:

1. Username: `john`
2. Password placeholder: `x`
3. UID: `1001`
4. Primary GID: `1001`
5. User information
6. Home directory: `/home/john`
7. Login shell: `/bin/bash`

The actual password hash is stored in `/etc/shadow`, not in `/etc/passwd`.

> **DEVOPS NOTE**
>
> Do not manually edit these files unless you understand the consequences. Use commands such as `useradd`, `usermod` and `passwd` to manage accounts safely.

3 Managing Linux Users

Linux provides commands to create, modify and delete user accounts.

Create a User

```
# Create a user
sudo useradd john

# Create a user with a home directory
sudo useradd -m john

# Set a password
sudo passwd john

# Create a user with a specific shell
sudo useradd -m -s /bin/bash john
```

> **INTERVIEW QUESTION**
>
> **What is the difference between `useradd` and `adduser`?**
>
> `useradd` is the low-level command available on all distributions; it does only what its options tell it to. On Debian and Ubuntu, `adduser` is a friendlier interactive script that creates the home directory and asks for a password. On RHEL-based systems, `adduser` simply points to `useradd`.

Modify a User

```
# Add a user to an additional group
sudo usermod -aG developers john

# Change a user's login shell
sudo usermod -s /bin/bash john

# Lock an account
sudo usermod -L john

# Unlock an account
sudo usermod -U john
```

INTERVIEW TRAP

`usermod -L` only locks **password** login. The user may still log in using an SSH key. To fully disable an account, also expire it or remove its login shell:

```
sudo usermod -L -e 1 john
sudo usermod -s /sbin/nologin john
```

Password Aging

```
# View password aging information
sudo chage -l john

# Force password change every 90 days
sudo chage -M 90 john
```

Delete a User

```
# Delete a user
sudo userdel john

# Delete a user and their home directory
sudo userdel -r john
```

Be careful with `-r`: it removes the user's home directory and mail spool where applicable.

4 Understanding Linux Groups

A group allows administrators to manage permissions for multiple users together.

For example, suppose a company has five developers who need access to `/opt/myapp`. Instead of granting permissions to each user separately, create a group named `developers` and add all five users.

```
# Create a group
sudo groupadd developers

# Add a user to a group
sudo usermod -aG developers john

# Check group membership
groups john

# View all groups
cat /etc/group

# Remove a user from a supplementary group
sudo gpasswd -d john developers

# Delete a group
sudo groupdel developers
```

IMPORTANT

The `-a` option in `usermod -aG` is important. Without `-a`, the user's existing supplementary group memberships can be replaced.

A user may need to log out and back in for new group membership to apply to their login session.

5 Understanding Linux File Permissions

Linux permissions control who can read, modify or execute a file.

```
# Check permissions
ls -l
```

Example:

```
-rwxr-xr-- 1 john developers 1024 Nov 5 app.sh
```

Breakdown:

```
- rwx r-x r--
  |   |   |
  |   |   └── Others permissions
  |   └────── Group permissions
  └────────── Owner permissions
```

The first character indicates the file type:

Symbol	Meaning
`-`	Regular file
`d`	Directory
`l`	Symbolic link
`c`	Character device
`b`	Block device

The remaining nine characters represent permissions.

Three Permission Types

Permission	File meaning	Directory meaning
Read (`r`)	Read file contents	List directory contents
Write (`w`)	Modify file contents	Create, delete or rename entries
Execute (`x`)	Execute the file	Enter/traverse the directory

> **IMPORTANT**
>
> Directory permissions behave differently from file permissions. For example, execute permission on a directory allows a user to traverse it, while read permission allows listing its entries.

6 Understanding Numeric Permissions

Linux represents permissions using numbers:

Permission	Number
Read (`r`)	4
Write (`w`)	2
Execute (`x`)	1

Add the values to calculate a permission number.

Permission	Calculation	Number
`rwx`	4 + 2 + 1	7
`rw-`	4 + 2	6
`r-x`	4 + 1	5
`r--`	4	4
`---`	No permission	0

```
chmod 755 app.sh
```

Means:

- Owner: `7` = read, write, execute
- Group: `5` = read, execute
- Others: `5` = read, execute

```
chmod 644 config.txt
```

Means:

- Owner: read and write
- Group: read only
- Others: read only

Common Permission Combinations

Permission	Typical use
`755`	Executable scripts and directories
`644`	Regular files and public-readable configuration
`600`	Private files accessible only by owner
`700`	Private directories or scripts
`775`	Shared directories for owner and group
`750`	Restricted group access

These are common patterns, not universal rules. Choose permissions based on the application's actual access requirements.

7 Using chmod to Change Permissions

The `chmod` command changes file and directory permissions.

Numeric Method

```
chmod 755 script.sh
chmod 644 config.txt
chmod 700 private_folder

# Apply permissions recursively
chmod -R 755 /opt/myapp
```

PRODUCTION WARNING

Be careful with recursive permission changes. Applying `755` to every file and directory can unintentionally make scripts executable and expose sensitive files.

PRODUCTION TIP

A safer approach is to set directories and files separately:

```
# Directories: 755
find /opt/myapp -type d -exec chmod 755 {} +

# Files: 644
find /opt/myapp -type f -exec chmod 644 {} +
```

Symbolic Method

You can also modify permissions using symbols:

Symbol	Meaning
`u`	Owner
`g`	Group
`o`	Others
`a`	All users
`+`	Add permission
`-`	Remove permission
`=`	Set exact permission

```
chmod u+x script.sh
chmod g+w shared.txt
chmod o-r secret.txt
chmod u=rwx,g=rx,o= file.sh
```

The symbolic method is useful when you want to change only one permission without replacing the complete permission set.

8 Understanding chown and chgrp

Permissions alone are not enough. Linux also tracks file ownership. Every file normally has:

- An owner user
- An owner group

```
# Check ownership
ls -l

# Change file owner
sudo chown john file.txt

# Change both owner and group
sudo chown john:developers file.txt

# Change only the group
sudo chgrp developers file.txt

# Change ownership recursively
sudo chown -R john:developers /opt/myapp
```

PRACTICAL DEVOPS EXAMPLE

Suppose your application runs using a Linux service account named `appuser`, and your application files are stored in `/opt/myapp`. You can assign ownership to the application user:

```
sudo chown -R appuser:appuser /opt/myapp
```

This helps ensure the application process can access its required files without running the application as root.

9 Special Permissions: SUID, SGID and Sticky Bit

Apart from regular read, write and execute permissions, Linux supports three special permission bits.

SUID (Set User ID)

SUID allows an executable file to run with the file owner's privileges rather than the privileges of the user executing it.

```
ls -l /usr/bin/passwd
```

You may see an `s` in the owner's execute position:

```
-rwsr-xr-x
```

The `passwd` command is a common example because it needs controlled access to password-related system files.

SGID (Set Group ID)

On an executable file, SGID allows the process to run with the file's group identity.

On a directory, newly created files and directories inherit the directory's group.

```
sudo chmod 2775 /opt/shared
```

This is useful for shared team directories where files should retain a common group.

Sticky Bit

The sticky bit on a directory restricts file deletion: users generally can delete or rename only their own files, the directory owner's files, or files when they have the relevant administrative privileges.

The `/tmp` directory commonly uses this permission:

```
ls -ld /tmp
```

```
drwxrwxrwt
```

```
# Set the sticky bit
sudo chmod +t /opt/shared
```

Special Permission Numbers

Special permissions are set using an extra digit in front of the normal three digits.

Permission	Value	Example	Result
SUID	4	`chmod 4755 file`	`-rwsr-xr-x`
SGID	2	`chmod 2775 dir`	`drwxrwsr-x`
Sticky bit	1	`chmod 1777 dir`	`drwxrwxrwt`

INTERVIEW TRAP

A lowercase `s` or `t` means the special bit and execute permission are both set. An uppercase `S` or `T` means the special bit is set but execute permission is missing, so it usually has no useful effect.

SECURITY TIP

SUID files are a common target for privilege escalation. Audit them regularly:

```
find / -perm -4000 -type f 2>/dev/null
```

Linux ignores the SUID bit on shell scripts for security reasons; it works only on compiled binaries.

Quick Comparison

Permission	Main purpose
SUID	Execute file with owner's identity
SGID	Execute file with group's identity; directories inherit group
Sticky bit	Restrict deletion of other users' files in shared directories

These permissions should be used carefully, especially on executable files.

10 Understanding umask

`umask` controls the default permissions removed when new files and directories are created.

```
# Check the current umask
umask
```

A common value is:

```
0022
```

Linux normally starts with these default permissions:

- Files: `666`
- Directories: `777`

With umask `022`:

Type	Default	Umask	Result
File	666	022	644
Directory	777	022	755

```
umask 027
```

New files typically get `640`, and new directories typically get `750`.

IMPORTANT

Umask removes permissions; it does not grant permissions. The final result also depends on the application or command creating the file.

INTERVIEW TRAP QUESTION

With umask `033`, what permission does a new file get?

`644`, not `633`. Umask is not normal subtraction; it removes permission bits. Files start without execute permission, so there is nothing to remove from the execute bits.

11 Access Control Lists (ACL)

Traditional Linux permissions provide access control for one owner, one group and others. But sometimes you need to grant access to a specific user without changing the main group. This is where ACLs are useful.

```
# Give user john read and write access to a file
sudo setfacl -m u:john:rw file.txt

# Give a group read and write access
sudo setfacl -m g:developers:rw file.txt

# View ACL permissions
getfacl file.txt

# Remove a user's ACL entry
sudo setfacl -x u:john file.txt

# Remove all extended ACL entries
sudo setfacl -b file.txt
```

HOW TO SPOT AN ACL

When a file has an ACL, `ls -l` shows a `+` at the end of the permission string, for example `-rw-rw-r--+`.

Practical Example

Suppose:

- `appuser` owns an application directory.
- `developers` is the development team.
- You want developers to read application logs without changing the directory's owner.

You can grant ACL access to the relevant log directory:

```
sudo setfacl -m g:developers:rx /var/log/myapp
```

You may also need appropriate permissions on parent directories and the log files themselves.

Default ACLs

A normal ACL applies only to existing files. New log files created later will not get access automatically. A **default ACL** on the directory makes new files inherit the rule:

```
sudo setfacl -d -m g:developers:rx /var/log/myapp
```

NOTE

ACL permissions can include a mask that limits effective permissions for named users and groups. Use `getfacl` to inspect the effective access.

12 Understanding sudo

The `sudo` command allows an authorized user to execute commands with elevated privileges.

```
# Run a command with elevated privileges
sudo systemctl restart nginx

# Check whether your user has sudo privileges
sudo -l
```

The sudo configuration is generally managed through `/etc/sudoers` and files inside `/etc/sudoers.d/`. Edit it safely using:

```
sudo visudo
```

`visudo` checks the configuration syntax before saving, helping prevent mistakes that could break sudo access.

Why sudo Is Important

Running everything as root is risky. Instead, administrators should use individual accounts and grant only the required administrative permissions.

For example, a developer may need permission to restart a particular service but should not have unrestricted root access. Sudo rules can be configured to allow specific commands, users or groups.

Example sudo Rules

```
# Create a separate rules file safely
sudo visudo -f /etc/sudoers.d/developers

# Allow john to restart nginx only
john ALL=(root) /usr/bin/systemctl restart nginx

# Allow all members of the developers group full sudo access
%developers ALL=(ALL) ALL
```

Granting Full Admin Access

The admin group name depends on the distribution:

```
# Ubuntu / Debian
sudo usermod -aG sudo john

# RHEL / Rocky / Amazon Linux
sudo usermod -aG wheel john
```

su vs sudo

Feature	su	sudo
Password required	Target user's password (usually root)	Your own password
Access level	Full shell as the target user	Specific commands, as configured
Logging	Limited	Every command is logged
Recommended	Rarely	Yes

sudo activity is logged in `/var/log/auth.log` on Ubuntu and `/var/log/secure` on RHEL-based systems.

13 Practical DevOps Scenarios

Scenario 1 **Permission denied while executing a script**

```
./deploy.sh
Permission denied
```

Check permissions:

```
ls -l deploy.sh
```

If execute permission is missing:

```
chmod +x deploy.sh
```

Scenario 2 Application cannot write to its log directory

Check ownership and permissions:

```
ls -ld /var/log/myapp
ls -l /var/log/myapp
```

Check which user runs the application:

```
ps aux | grep myapp
```

Then grant only the required access, for example by correcting ownership:

```
sudo chown -R appuser:appuser /var/log/myapp
```

Do not immediately use `chmod 777`. First identify which user needs access and why.

Scenario 3 Developer needs access to application files

Create a shared group:

```
sudo groupadd developers
sudo usermod -aG developers john
```

Set the directory's group and permissions:

```
sudo chown -R appuser:developers /opt/myapp
sudo chmod -R 775 /opt/myapp
```

For a production application, avoid applying the same permissions blindly to all files. Consider SGID on shared directories and separate read-only access from write access where possible.

Scenario 4 Find why a user cannot access a file

```
id john
ls -l file.txt
getfacl file.txt
```

Also check execute permission on every parent directory in the path. A user needs directory traversal permission to reach a file.

If permissions and ACLs look correct on a RHEL-based system, check the SELinux context using `ls -Z file.txt`.

Scenario 5 Even root cannot delete a file

```
rm config.txt
rm: cannot remove 'config.txt': Operation not permitted
```

Check the file's attributes:

```
lsattr config.txt
----i---------------- config.txt
```

The `i` means the file is **immutable**. Nobody, not even root, can modify or delete it until the attribute is removed:

```
sudo chattr -i config.txt
```

Administrators sometimes use `chattr +i` to protect critical configuration files from accidental changes.

14 Common Interview Questions

Q1. What is the difference between root and a regular user?

Root has UID 0 and broad administrative privileges. Regular users have limited access based on permissions and sudo rules.

Q2. What is the difference between UID and GID?

UID identifies a user, while GID identifies a group.

Q3. What is the difference between `/etc/passwd` and `/etc/shadow`?

`/etc/passwd` contains account information. `/etc/shadow` stores password hashes and password-aging information.

Q4. What does `chmod 755` mean?

Owner gets read, write and execute. Group and others get read and execute.

Q5. What is the difference between chmod and chown?

`chmod` changes permissions. `chown` changes ownership.

Q6. Why should we avoid chmod 777?

It grants read, write and execute permissions to everyone, potentially allowing unauthorized modification or execution.

Q7. What is the difference between SUID and SGID?

SUID uses the executable file owner's identity. SGID uses the executable file's group identity. On directories, SGID makes new entries inherit the directory's group.

Q8. What is umask?

Umask controls which permissions are removed from the default permissions when new files and directories are created.

Q9. What is ACL?

ACL provides more detailed permissions, allowing access to specific users or groups beyond the standard owner/group/others model.

Q10. What is the purpose of sudo?

It allows authorized users to run commands with elevated privileges without logging in as root.

Q11. Why can a user not access a file even when the file has read permission?

The user may lack execute (traverse) permission on one of the parent directories, or an ACL or other access-control mechanism may restrict access.

Q12. How do you troubleshoot a permission denied error?

Check the current user, groups, file ownership, permissions, ACL entries and parent directory permissions.

Q13. What is the difference between su and sudo?

`su` switches to another user (usually root) using that user's password and gives a full shell. `sudo` runs specific commands with elevated privileges using your own password, and every command is logged. sudo is safer and preferred.

Q14. How can you protect a file from being deleted, even by root?

Make it immutable using `chattr +i file`. Check attributes with `lsattr`, and remove the protection with `chattr -i file`.

15 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ Linux users, root accounts, UID and GID.
- ✓ User and group management commands.
- ✓ Important account files such as `/etc/passwd` and `/etc/shadow`.
- ✓ File and directory permissions.
- ✓ Numeric and symbolic `chmod`.
- ✓ Ownership using `chown` and `chgrp`.
- ✓ SUID, SGID and sticky bit, including their numeric values.
- ✓ Default permissions using umask.
- ✓ ACL-based access control, including default ACLs.
- ✓ Administrative access using sudo, and su vs sudo.
- ✓ Immutable files using `chattr`.
- ✓ Troubleshooting common permission errors.

PRACTICAL EXERCISES

1. Create a user named `devuser`.
2. Create a group named `developers`.
3. Add `devuser` to the `developers` group.
4. Create `/opt/project` and assign it to `devuser:developers`.
5. Set permissions to `775` and verify using `ls -ld`.
6. Create a file and set its permissions to `640`.
7. Give another user read access using ACL.
8. Check the current umask and create a new file to inspect its permissions.
9. Find the SUID permission on `/usr/bin/passwd`.
10. Use `sudo -l` to inspect your administrative privileges.
11. Make a file immutable with `chattr +i`, try to delete it, then remove the attribute.

NEXT CHAPTER

Chapter 04: Linux Package Management and Software Installation

You will learn how Linux packages work, how to install and update software using APT and DNF, manage repositories, and troubleshoot package installation problems.

END OF CHAPTER 3